

Graphviz has its own dialect that you will have to learn. The language is simple, elegant and powerful. The good thing about Graphviz is that you can write its code using a simple plain text editor—a wonderful side effect of this is that you can easily write UNIX scripts that generate Graphviz code. In fact, this article has such a script that is written in Perl—my favourite scripting language.

Graphviz comprises the following programs and libraries:

- The **dot** program: A utility program for drawing directed graphs. It accepts input in the dot language. The dot language can define three kinds of objects: graphs, nodes and edges. It uses the Sugiyama-style hierarchical layout.
- The **neato** program: This is a utility program for drawing undirected graphs. This kind of graph is commonly used for telecommunications and computer programming tasks. Neato uses an implementation of the Kamada-Kawai algorithm for symmetric layouts.
- The **twopi** program: This is a utility program for drawing graphs using a circular layout. One node is chosen as the centre, and the other nodes are placed around the centre in a circular pattern. If a node is connected to the centre node, it is placed at distance 1. If a node is connected to a node directly connected to the centre node, it is placed at distance 2 and so on.
- **Dotty, tcldot and lefty**: These are three graphical programs. Dotty is a customisable interface for the X Window System written in lefty. Tcldot is a customisable graphical interface written in Tcl 7. Lefty is a graphics editor for technical pictures.
- **Libgraph and libagraph**: These are the drawing libraries. Their presence means an application can use Graphviz as a library rather than as a software tool.
- **circo**: This is a utility program for creating a circular layout of graphs.
- **fdp**: A utility program for generating undirected graphs.
- **sfdp**: A utility program for constructing large undirected graphs.

Table 1 shows the Node attributes and Table 2 shows the Edge attributes.

You are now ready to continue with practical examples. Please do not forget to experiment, make changes and create your own graphs as you are reading the article.

A Graphviz example

A database schema, visualised with the help of Graphviz, will

Table 1: Node attributes

Name	Explanation	Allowed Values
shape	The shape of the node	ellipse, diamond, box, circle, etc.
height	The height in inches	a number
width	The width in inches	a number
label	The name of the node	alphanumeric
fontsize	The size of the font	a number
fontname	The name of the font	Courier, Helvetica, Times—Roman
fontcolour	The colour of the font	White, black, blue, etc.
style	The style name	bold, dotted, filled, etc
colour	The colour of the node	white, black, etc.
pos	shape The coordinates of the position	

Table 2: Edge attributes

Name	Explanation	Allowed Values
label	The label of the edge	alphanumeric
fontsize	The size of the font	
fontname	The name of the font	
fontcolour	The colour of the font	
style	The style name	bold, dotted, filled, etc.
colour	The colour of the edge	white, black, blue, etc
len	The length of the edge	
dir	The direction of the edge	forward, back, both or none
decorate	Draws a line that connects labels with their edges	o or 1
id	Option value to denote different edges	alphanumeric

be presented in this part. The presented schema is simple; yet you can still understand how elegant it is. By reading the Graphviz code you can understand that lines beginning with the # character are comments.

The Graphviz code for creating Figure 1 is as follows:

```
digraph G
{
    graph [rankdir = "LR"];
    node[fontsize = "14" style=bold];

    # Table-field connection part.
    BONUS [label="<tb> BONUS | sal | comm | ename | job"
    shape = "record"];
    DEPT [label="<tb> DEPT | loc | dname | deptno"
    shape = "record"];
    EMP [label="<tb> EMP | empno | ename | comm | mgr |
    hiredate | deptno | job"
    shape = "record"];
    CLIENT [label="<tb> CLIENT | sal | comm | ename | job"
    shape = "record"];
    CLERK [label="<tb> CLERK | sal | comm | ename | job"
    shape = "record"];
    ORDER [label="<tb> ORDER | sal | comm | ename | job"
    shape = "record"];
    FOO [label="<tb> FOO | sal | comm | ename | job"]
}
```